FUNDAMENTALS OF QUANTUM COMPUTING

UNIT III — QUANTUM ALGORITHMS

Lecture Notes | 5 Hours

Course progression: Quantum foundations → Quantum gates & circuits → Quantum algorithms → Programming, simulation & error correction → Applications & emerging trends

UNIT III — QUANTUM ALGORITHMS

This unit introduces the major algorithmic ideas that distinguish quantum computing from classical computation. The emphasis is on how superposition, interference, phase, and entanglement are converted into useful computational procedures.

Unit Syllabus Coverage

- Quantum algorithms: motivation and algorithmic advantage
- Deutsch and Deutsch–Jozsa algorithms
- Bernstein–Vazirani algorithm
- Simon's algorithm
- Quantum Fourier Transform (QFT)
- Quantum phase estimation
- Grover's search algorithm
- Amplitude amplification
- Shor's factoring algorithm: high-level structure
- Variational and hybrid quantum algorithms
- Worked circuits, simulation, and AI-assisted algorithm analysis

Learning Outcomes

- Explain why quantum algorithms use interference rather than simply relying on superposition.
- Trace the major steps of foundational oracle-based algorithms.
- Construct and interpret simple Deutsch–Jozsa and Bernstein–Vazirani circuits.
- Explain the role of the Quantum Fourier Transform and phase estimation.
- Describe Grover search and amplitude amplification at circuit level.
- Explain the high-level relationship between period finding and Shor's algorithm.
- Distinguish between fault-tolerant algorithms and near-term hybrid/variational methods.
- Use a quantum simulator to inspect probabilities and measurement outcomes.

Part A: Introduction to Quantum Algorithms

A quantum algorithm is a sequence of quantum operations and measurements designed to solve a computational problem. A useful quantum algorithm does not obtain an advantage merely because a qubit can be in a superposition. The algorithm must arrange amplitudes so that useful outcomes are amplified while unwanted outcomes interfere destructively.

Why Quantum Algorithms Are Different

- Superposition provides a state space containing many computational basis states.
- Entanglement can create correlations that cannot be represented as independent classical probabilities.
- Interference controls which computational paths become more or less likely.
- Measurement converts the final quantum state into classical information.
- Oracle-based algorithms often encode a problem into a reversible quantum operation.

Algorithmic Workflow

1. Prepare one or more qubits in a known initial state.
2. Create the required superposition or structured quantum state.
3. Apply problem-specific operations, often represented by an oracle.
4. Use interference, phase manipulation, or amplitude amplification.
5. Measure the final state and interpret the classical result.

Oracle Concept

An oracle is a black-box function or reversible operation used by an algorithm. In introductory quantum algorithms, the oracle separates the computational problem from the quantum procedure. The same quantum framework can then be analyzed for different hidden functions.

$$|x\rangle|y\rangle \rightarrow |x\rangle|y \oplus f(x)\rangle$$

Here, x is the input register, y is an auxiliary register, f(x) is the function being queried, and $\oplus$ denotes XOR.

Part B: Deutsch's Algorithm

Deutsch's algorithm determines whether a one-bit Boolean function is constant or balanced using a single query to the function oracle. Classically, the two possible inputs may need to be evaluated separately in the deterministic setting. The quantum procedure uses phase kickback and interference to distinguish the cases.

Problem Definition

For $f:\{0,1\} \rightarrow \{0,1\}$, the function is either constant, meaning $f(0)=f(1)$, or balanced, meaning $f(0) \neq f(1)$.

Circuit Idea

- Initialize the two-qubit state as $|0\rangle|1\rangle$.
- Apply H to both qubits.
- Apply the oracle U_f.
- Apply H to the first qubit.
- Measure the first qubit.
- Result 0 indicates constant; result 1 indicates balanced.

Phase Kickback

The second qubit is prepared in $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$. The oracle then converts the function value into a phase on the input state. This is a central technique reused in several quantum algorithms.

$$U_f(|x\rangle|-\rangle) = (-1)^{f(x)} |x\rangle|-\rangle$$

Worked Example — Deutsch Algorithm

Let $f(0)=0$ and $f(1)=1$. This function is balanced. After the first Hadamard layer, the input qubit contains equal amplitudes. The oracle introduces opposite phases for the two inputs. The final Hadamard converts this phase difference into the computational result $|1\rangle$, so measurement identifies the function as balanced.

Part C: Deutsch–Jozsa Algorithm

The Deutsch–Jozsa algorithm generalizes the same idea to an n-bit input function. The promise is that the function is either constant or balanced.

Problem Statement

Given $f:\{0,1\}^n \rightarrow \{0,1\}$, determine whether f is constant or balanced, assuming the promise is valid.

Algorithm Steps

6. Prepare n input qubits in $|0\rangle$ and one auxiliary qubit in $|1\rangle$.
7. Apply Hadamard gates to all qubits.

8. Apply the oracle U_f.
9. Apply Hadamard gates to the n input qubits.
10. Measure the input register.

Interpretation

- All zeros in the input measurement correspond to the constant case.
- Any nonzero bit string indicates the balanced case.
- The algorithm obtains the promised classification with one oracle query.

Circuit-Level Insight

The oracle does not directly reveal every value of f(x). Instead, it changes phases across a superposition. The final Hadamard transform makes the global property of the function observable.

Part D: Bernstein–Vazirani Algorithm

The Bernstein–Vazirani algorithm identifies a hidden n-bit string using a single oracle query. The hidden string is encoded in a linear Boolean function.

Problem Definition

$$f_s(x) = s \cdot x \bmod 2$$

The unknown string s is the quantity to recover, and x is an n-bit input string.

Algorithm Steps

- Prepare n input qubits in $|0\rangle$ and one auxiliary qubit in $|1\rangle$.
- Apply H gates to create an equal superposition.
- Apply the oracle for f_s.
- Use phase kickback to encode s into the phases of the input register.
- Apply H gates again.
- Measure the input register to obtain s.

Worked Example

Suppose the hidden string is s=1011. The oracle computes the parity function s·x mod 2. After phase kickback and the final Hadamard transform, the input register ideally collapses to $|1011\rangle$ upon measurement.

Part E: Simon's Algorithm

Simon's algorithm is an important early example of a quantum algorithm with an exponential separation between its quantum query complexity and the corresponding classical randomized query complexity under the standard oracle model.

Problem Setup

The oracle implements a function f:{0,1}^n→{0,1}^n with the promise that there exists a hidden nonzero string s such that:

$$f(x) = f(x \oplus s)$$

The goal is to determine the hidden period s.

Algorithm Outline

11. Initialize two n-qubit registers.
12. Create a uniform superposition over the first register.
13. Query the oracle.
14. Measure or discard the second register, leaving a superposition related to x and x⊕s.
15. Apply Hadamard gates to the first register.
16. Measure to obtain a bit string y satisfying y·s = 0 mod 2.
17. Repeat enough times to collect independent equations.
18. Solve the resulting linear system over GF(2) to recover s.

Key Insight

The quantum part produces linear constraints on the hidden string. Classical linear algebra is then used to combine those constraints and determine the answer.

Part F: Quantum Fourier Transform

The Quantum Fourier Transform is the quantum analogue of the discrete Fourier transform. It maps computational-basis amplitudes into a Fourier basis and is a central subroutine in phase estimation and period-finding algorithms.

Definition

$$QFT|x\rangle = (1/\sqrt{N}) \Sigma_{y=0}^{N-1} e^{2\pi i x y/N}|y\rangle$$

For an n-qubit register, N=2^n.

Important Properties

- The QFT is unitary and therefore reversible.
- It transforms information encoded in phases into measurable interference patterns.
- The inverse QFT is obtained by reversing the circuit and conjugating the phase rotations.
- Efficient quantum circuits implement the QFT using Hadamard and controlled phase-rotation gates.

QFT Circuit Structure

19. Apply a Hadamard gate to the first qubit.
20. Apply controlled phase rotations from later qubits.
21. Repeat the pattern for subsequent qubits.
22. Reverse the qubit order using SWAP gates when required by the chosen circuit convention.

Worked Example — Two-Qubit QFT

For a two-qubit register, the QFT can be constructed using a Hadamard on one qubit, a controlled phase rotation, a Hadamard on the other qubit, and a final SWAP. The circuit converts relative phase information into amplitudes in the Fourier basis.

Part G: Quantum Phase Estimation

Quantum Phase Estimation (QPE) estimates an eigenphase associated with a unitary operator. It is one of the most important building blocks in quantum algorithms for simulation, factoring, and eigenvalue problems.

Eigenvalue Relationship

$$U|\psi\rangle = e^{2\pi i \varphi}|\psi\rangle$$

The goal is to estimate φ. The input state |ψ⟩ is assumed to be an eigenstate of U.

QPE Procedure

23. Prepare a phase-estimation register in |0...0⟩.
24. Apply Hadamard gates to create a uniform superposition.
25. Apply controlled powers U, U^2, U^4, ... to encode phase information.
26. Apply the inverse QFT to the estimation register.
27. Measure the estimation register to obtain an approximation to φ.

Accuracy and Resources

Using more estimation qubits generally permits a finer representation of the phase. Practical implementations must also account for circuit depth, controlled-unitary cost, noise, and the quality of the input eigenstate.

Part H: Grover's Search Algorithm

Grover's algorithm searches an unstructured database or solution space. For N possible items with a suitable oracle, the algorithm requires O(√N) oracle queries, compared with O(N) queries for classical exhaustive search in the black-box model.

Core Components

- Uniform initialization of the search register.
- Oracle that marks target states with a phase change.
- Diffusion operator that performs amplitude amplification.
- Repeated Grover iterations.
- Measurement of the amplified state.

Grover Oracle

The oracle marks a solution by applying a phase flip to its amplitude.

$$|x\rangle \rightarrow (-1)^{f(x)}|x\rangle$$

Diffusion Operator

The diffusion operation reflects amplitudes about their average. Combined with the oracle, this transfers probability amplitude from non-solution states toward marked states.

Grover Iteration

28. Apply the oracle.
29. Apply the diffusion operator.
30. Repeat for an appropriate number of iterations.
31. Measure the search register.

Worked Example — Four-Item Search

For four equally likely candidates and one marked item, initialize two qubits with H⊗H. One Grover iteration consists of an oracle marking the target followed by the diffusion operation. Ideally, the target probability becomes 1 before measurement.

Part I: Amplitude Amplification

Amplitude amplification generalizes the central idea of Grover search. If a quantum procedure prepares a state in which the desired outcomes have some nonzero success probability, amplitude amplification can increase that probability using repeated reflections.

Geometric View

- The state is decomposed into a good subspace and a bad subspace.
- The oracle changes the phase of the good component.
- A second reflection changes the angle toward the good subspace.
- Repeated iterations rotate the state toward the desired outcomes.

This framework is useful beyond database search and appears in several quantum speedup constructions.

Part J: Shor's Factoring Algorithm

Shor's algorithm is a major quantum algorithm for integer factorization. Its key quantum component is period finding, implemented using phase estimation/QFT ideas.

High-Level Structure

32. Choose a suitable integer a relatively prime to the number N being factored.
33. Consider the periodic function f(x)=a^x mod N.
34. Use a quantum period-finding procedure to estimate the period r.
35. Use classical number-theoretic post-processing to obtain factors of N when the required conditions hold.

Role of the QFT

The QFT helps extract periodic information from the quantum state. The measured result provides an approximation from which the period can be recovered using classical continued-fraction techniques.

Important Concept

Shor's algorithm does not simply 'try all factors at once.' Its speedup comes from encoding periodic structure into a quantum state and extracting that structure efficiently.

Part K: Variational and Hybrid Quantum Algorithms

Near-term quantum computing often uses hybrid algorithms in which a classical optimizer controls parameters of a quantum circuit. These methods are designed for settings where deep fault-tolerant circuits are not yet practical.

Variational Quantum Eigensolver (VQE)

VQE estimates the ground-state energy of a quantum system by preparing a parameterized trial state and minimizing the measured expectation value of a Hamiltonian.

$$E(\theta) = \langle \psi(\theta) | H | \psi(\theta) \rangle$$

VQE Workflow

36. Choose a parameterized quantum circuit (ansatz).
37. Initialize its parameters.
38. Run the circuit and measure required observables.
39. Estimate the expectation value of the Hamiltonian.
40. Use a classical optimizer to update parameters.

41. Repeat until the objective converges or a stopping condition is reached.

Quantum Approximate Optimization Algorithm (QAOA)

QAOA is a parameterized circuit approach for combinatorial optimization. Alternating problem-dependent and mixing operations create a parameterized state, while a classical optimizer searches for useful parameters.

Hybrid Algorithm Pattern

- Classical computer: parameter selection and optimization.
- Quantum processor/simulator: state preparation and measurement.
- Classical post-processing: objective evaluation and stopping criteria.

Part L: Algorithm Comparison

Conceptual Comparison

Algorithm	Main Problem	Core Quantum Idea	Key Tool
Deutsch	Constant vs balanced one-bit function	Phase kickback + interference	Hadamard
Deutsch–Jozsa	Constant vs balanced n-bit function	Global phase structure	Oracle + H
Bernstein–Vazirani	Recover hidden bit string	Phase encoding	Oracle + H
Simon	Find hidden XOR period	Entangled pair structure	H + linear algebra
Grover	Unstructured search	Amplitude amplification	Oracle + diffusion
QPE	Estimate eigenphase	Phase-to-amplitude conversion	Controlled-U + inverse QFT
Shor	Integer factoring	Period finding	QFT/QPE ideas
VQE	Ground-state energy	Variational optimization	Parameterized circuit
QAOA	Combinatorial optimization	Alternating operators	Hybrid optimization

Part M: Worked Circuit Designs and Simulation

Exercise 1 — Deutsch–Jozsa Circuit

Build a two-input Deutsch–Jozsa circuit for a chosen promised constant or balanced function. Run several shots and compare the observed histogram with the theoretical result.

Exercise 2 — Bernstein–Vazirani

Choose a hidden string such as 1101. Construct the oracle and verify that the final measurement returns the hidden string with high probability in an ideal simulator.

Exercise 3 — Grover Search

Construct a two-qubit search circuit with one marked state. Compare the probability distribution before and after one Grover iteration.

Exercise 4 — QFT

Prepare a simple basis state and apply a QFT circuit. Inspect the amplitudes/probabilities and then apply the inverse QFT to verify reversibility.

Exercise 5 — QPE

Use a simple unitary with a known eigenphase. Simulate controlled powers of the unitary, apply inverse QFT, and compare the measured phase estimate with the expected value.

Part N: AI-Assisted Quantum Algorithm Analysis

AI tools can support learning and debugging, but the quantum result should be verified using mathematical reasoning or a trusted simulator.

- Ask an AI system to explain each circuit layer and predict the state after selected gates.
- Generate test cases for oracle functions and compare predicted and simulated outputs.
- Use AI to identify redundant gates or suggest circuit decompositions, then verify equivalence.
- Ask for symbolic matrix multiplication and check the result independently.
- Compare ideal simulation results with noisy simulation in later units.

Verification Checklist

- Check qubit count and register ordering.
- Confirm the oracle is unitary/reversible.
- Check whether the algorithm assumes a promise condition.
- Verify expected probabilities analytically for small examples.
- Run enough shots before interpreting a histogram.
- Distinguish a simulator result from a proof of algorithmic correctness.

Part O: Integrated Worked Examples

Worked Example 1 — Why H Alone Is Not an Algorithm

Applying H to $|0\rangle$ creates $(|0\rangle+|1\rangle)/\sqrt{2}$. Measuring gives 0 or 1 with equal probability. This demonstrates superposition, but it does not solve a computational problem. A useful algorithm must subsequently manipulate amplitudes and phases so that measurement contains problem-specific information.

Worked Example 2 — Grover Iteration Concept

Start from a uniform state. The oracle flips the phase of the marked state. The diffusion operation reflects all amplitudes about their average. Together these two reflections rotate the state toward the marked subspace. Repeating too many times can reduce the success probability, so iteration count matters.

Worked Example 3 — From Phase to Measurement

Suppose a unitary has eigenvalue $e^{2\pi i\varphi}$. Phase estimation stores information about φ in relative phases. Controlled powers of the unitary increase the phase sensitivity. The inverse QFT converts the phase pattern into a computational-basis distribution that can be measured.

Worked Example 4 — Hybrid Optimization

A parameterized circuit prepares $|\psi(\theta)\rangle$. Measurements estimate an objective such as $\langle H\rangle$. A classical optimizer changes θ and repeats the experiment. The quantum circuit and classical optimizer therefore form one computational loop.

Unit Summary

- Quantum algorithms exploit interference and phase, not superposition alone.

- Deutsch, Deutsch–Jozsa, and Bernstein–Vazirani illustrate oracle-based phase-kickback techniques.
- Simon's algorithm demonstrates how quantum measurements can generate equations about hidden structure.
- The QFT transforms phase information and is central to phase estimation and period finding.
- QPE estimates eigenphases using controlled powers of a unitary and the inverse QFT.
- Grover's algorithm uses amplitude amplification to obtain a quadratic query improvement for unstructured search.
- Shor's algorithm uses quantum period finding as its central quantum subroutine.
- VQE and QAOA illustrate hybrid quantum-classical computation for near-term devices.
- Simulation is valuable for understanding algorithms, but circuit conventions, noise, and measurement statistics must be checked carefully.

Review Questions

42. What makes a quantum algorithm different from simply preparing a superposition?
43. Define an oracle in the context of quantum algorithms.
44. State the problem solved by Deutsch's algorithm.
45. Explain phase kickback.
46. What is the promise in the Deutsch–Jozsa problem?
47. How does Bernstein–Vazirani recover a hidden bit string?
48. Write the defining promise of Simon's problem.
49. Why is classical linear algebra used after Simon measurements?
50. Define the Quantum Fourier Transform.
51. Why is the QFT important in quantum algorithms?
52. What is an eigenstate and eigenphase in QPE?
53. Outline the steps of Quantum Phase Estimation.
54. What is the purpose of the inverse QFT in QPE?
55. State the query complexity of Grover search in the black-box model.
56. What does the Grover oracle do?
57. Explain the role of the diffusion operator.
58. What is amplitude amplification?
59. What is the central computational task in Shor's algorithm?
60. How does period finding relate to factoring?
61. What is a variational quantum algorithm?
62. Explain the VQE objective function.
63. What is QAOA used for?
64. Compare fault-tolerant and near-term algorithmic approaches.
65. Why can excessive Grover iterations reduce the probability of finding the target?
66. Why is measurement repeated for practical quantum algorithms?
67. How can an AI assistant help analyze a quantum circuit?
68. Why should AI-generated quantum-circuit suggestions be independently verified?

Self-Assessment Checklist

- I can explain the role of superposition, phase, and interference in quantum algorithms.
- I can trace a basic Deutsch or Deutsch–Jozsa circuit.
- I can explain how phase kickback works.
- I can describe Bernstein–Vazirani and Simon's algorithms.
- I can state the purpose and structure of the QFT.
- I can explain the major steps of phase estimation.

- I can describe Grover’s oracle and diffusion operator.
- I can explain the high-level structure of Shor’s algorithm.
- I can distinguish VQE and QAOA from fixed, fault-tolerant algorithms.
- I can validate small circuits using an ideal simulator.

Suggested Mini-Lab

Title: Exploring Foundational Quantum Algorithms with a Simulator

69. Implement Deutsch–Jozsa for a small input size.
70. Implement Bernstein–Vazirani for a selected hidden string.
71. Implement a two-qubit Grover search.
72. Implement a small QFT and its inverse.
73. Inspect state vectors and measurement histograms.
74. Record circuit depth and gate counts.
75. Repeat selected circuits under a simple noise model if the simulator supports it.
76. Write a short comparison of ideal and noisy results.

Expected Output: Circuit diagrams, measurement histograms, brief observations, and a one-page conclusion explaining where interference or amplitude amplification appears in each algorithm.